

EXPERIMENT 6:

6. Construct a C program to implement preemptive priority scheduling algorithm

```

1 #include <stdio.h>
2 int main() {
3     int i, NOP, sum = 0, count = 0, y, quant, wt = 0, tat = 0;
4     int at[10], bt[10], temp[10];
5     float avg_wt, avg_tat;
6     printf("Total number of processes in the system: ");
7     scanf("%d", &NOP);
8     y = NOP;
9     for (i = 0; i < NOP; i++) {
10        printf("\nEnter the Arrival and Burst time of Process[%d]\n", i + 1);
11        printf("Arrival time is: \t");
12        scanf("%d", &at[i]);
13        printf("Burst time is: \t");
14        scanf("%d", &bt[i]);
15        temp[i] = bt[i];
16    }
17    printf("\nEnter the Time Quantum for the process: \t");
18    scanf("%d", &quant);
19    printf("\nProcess No \t\t Burst Time \t TAT \t\t Waiting Time");
20    for (sum = 0, i = 0; y != 0; ) {
21        if (temp[i] <= quant && temp[i] > 0) {
22            sum = sum + temp[i];
23            temp[i] = 0;
24            count = 1;
25        }
26        else if (temp[i] > 0) {
27            temp[i] = temp[i] - quant;
28            sum = sum + quant;
29        }
30        if (temp[i] == 0 && count == 1) {
31            y--;
32            printf("\nProcess[%d] \t\t %d \t\t %d \t\t %d", i + 1, bt[i], sum - at[i], sum - at[i] - bt[i]);
33            wt = wt + sum - at[i] - bt[i];
34            tat = tat + sum - at[i];
35            count = 0;
36        }
37        if (i == NOP - 1) {
38            i = 0;
39        } else if (at[i + 1] <= sum) {
40            i++;
41        } else {
42            i = 0;
43        }
44    }
45    avg_wt = (float)wt / NOP;
46    avg_tat = (float)tat / NOP;
47    printf("\n\nAverage Waiting Time: \t%f", avg_wt);
48    printf("\n\nAverage Turnaround Time: \t%f\n", avg_tat);
49    return 0;
50 }

```

Status: Accepted

Memory Used: 716 byte Execution Time: 0.004 sec

Total number of processes in the system:
Enter the Arrival and Burst time of Process[1]
Arrival time is: Burst time is:
Enter the Arrival and Burst time of Process[2]
Arrival time is: Burst time is:
Enter the Time Quantum for the process:

Process No	Burst Time	TAT	Waiting Time
Process[2]	4	7	3
Process[1]	5	9	4

Average Waiting Time: 3.500000
Average Turnaround Time: 8.000000

Input

2 0 5 1 4 2

EXPERIMENT 7:

7. Construct a C program to implement a non-preemptive SJF algorithm.

Online C Compiler

Share

Run

```
1 #include <stdio.h>
2 int main() {
3     int at[10], bt[10], pr[10];
4     int n, i, j, temp, time = 0, count, over = 0, sum_wait = 0, sum_turnaround = 0, start;
5     float avgwait, avgturn;
6     printf("Enter the number of processes\n");
7     scanf("%d", &n);
8     for (i = 0; i < n; i++) {
9         printf("Enter the arrival time and execution time for process %d\n", i + 1);
10        scanf("%d%d", &at[i], &bt[i]);
11        pr[i] = i + 1;
12    }
13    for (i = 0; i < n - 1; i++) {
14        for (j = i + 1; j < n; j++) {
15            if (at[i] > at[j]) {
16                temp = at[i];
17                at[i] = at[j];
18                at[j] = temp;
19                temp = bt[i];
20                bt[i] = bt[j];
21                bt[j] = temp;
22                temp = pr[i];
23                pr[i] = pr[j];
24                pr[j] = temp;
25            }
26        }
27    }
28    printf("\n\nProcess\tArrival time\tExecution time\tStart time\tEnd time\tWaiting time\tTurnaround time\n\n");
29    while (over < n) {
30        count = 0;
31        if (time < at[over]) {
32            time = at[over];
33        }
34        for (i = over; i < n; i++) {
35            if (at[i] <= time) {
36                count++;
37            }
38            else {
39                break;
40            }
41        }
42        if (count > 1) {
43            for (i = over; i < over + count - 1; i++) {
44                for (j = i + 1; j < over + count; j++) {
45                    if (bt[i] > bt[j]) {
46                        temp = at[i];
47                        at[i] = at[j];
48                        at[j] = temp;
49                        temp = bt[i];
50                        bt[i] = bt[j];
51                        bt[j] = temp;
52                        temp = pr[i];
53                        pr[i] = pr[j];
54                        pr[j] = temp;
55                    }
56                }
57            }
58            start = time;
59            time += bt[over];
```

Status: Accepted

Memory Used: 720 byte Execution Time: 0.004 sec

Enter the number of processes
Enter the arrival time and execution time for process 1
Enter the arrival time and execution time for process 2
Enter the arrival time and execution time for process 3

Process	Arrival time	Execution time	Start time	End time	Waiting time
p[1]	1	3	1	4	0
p[2]	2	6	4	10	2
p[3]	3	8	10	18	7

Average waiting time is 3.000000
Average turnaround time is 8.666667

Input

3
13
26
38

8. Construct a C program to simulate Round Robin scheduling algorithm with C.

[illegible]

EXPERIMENT 9:

9. Illustrate the concept of inter-process communication using shared memory with a C program

Online C Compiler

Share

Run

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/ipc.h>
6 #include <sys/shm.h>
7
8 #define SHM_SIZE 1024
9
10 int main() {
11     key_t key = ftok("shmfile", 65);
12
13     int shmid = shmget(key, SHM_SIZE, IPC_CREAT | 0666);
14     if (shmid == -1) {
15         perror("shmget");
16         exit(EXIT_FAILURE);
17     }
18
19     char *shm_ptr = (char*)shmat(shmid, NULL, 0);
20     if (shm_ptr == (char*)(-1)) {
21         perror("shmat");
22         exit(EXIT_FAILURE);
23     }
24
25     strcpy(shm_ptr, "Hello, shared memory!");
26
27     printf("Data written to shared memory: %s\n", shm_ptr);
28
29     if (shmdt(shm_ptr) == -1) {
30         perror("shmdt");
31         exit(EXIT_FAILURE);
32     }
33
34     if (shmctl(shmid, IPC_RMID, NULL) == -1) {
35         perror("shmctl");
36         exit(EXIT_FAILURE);
37     }
38
39     return 0;
40 }
```

Output

Status: **Accepted**

Memory Used: 756 byte | Execution Time: 0.003 sec

Data written to shared memory: Hello, shared memory!

Input

Enter your input here...

EXPERIMENT 10:

Illustrate the concept of inter-process communication using message queue with a C program

Online C Compiler

Share

Run

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 #include <sys/ipc.h>
7 #include <sys/msg.h>
8
9 struct message {
10     long msg_type;
11     char msg_text[100];
12 };
13
14 int main() {
15     key_t key = ftok("msgqfile", 65);
16
17     int msgid = msgget(key, IPC_CREAT | 0666);
18     if (msgid == -1) {
19         perror("msgget");
20         exit(EXIT_FAILURE);
21     }
22
23     struct message msg;
24     msg.msg_type = 1;
25
26     strcpy(msg.msg_text, "Hello, message queue!");
27
28     if (msgsnd(msgid, (void*)&msg, sizeof(msg.msg_text), IPC_NOWAIT) == -1) {
29         perror("msgsnd");
30         exit(EXIT_FAILURE);
31     }
32
33     printf("Producer: Data sent to message queue: %s\n", msg.msg_text);
34
35     if (msgrcv(msgid, (void*)&msg, sizeof(msg.msg_text), 1, 0) == -1) {
36         perror("msgrcv");
37         exit(EXIT_FAILURE);
38     }
39
40     printf("Consumer: Data received from message queue: %s\n", msg.msg_text);
41
42     if (msgctl(msgid, IPC_RMID, NULL) == -1) {
43         perror("msgctl");
44         exit(EXIT_FAILURE);
45     }
46
47     return 0;
48 }
```

Output

Status: Accepted

Memory Used: 732 byte Execution Time: 0.003 sec

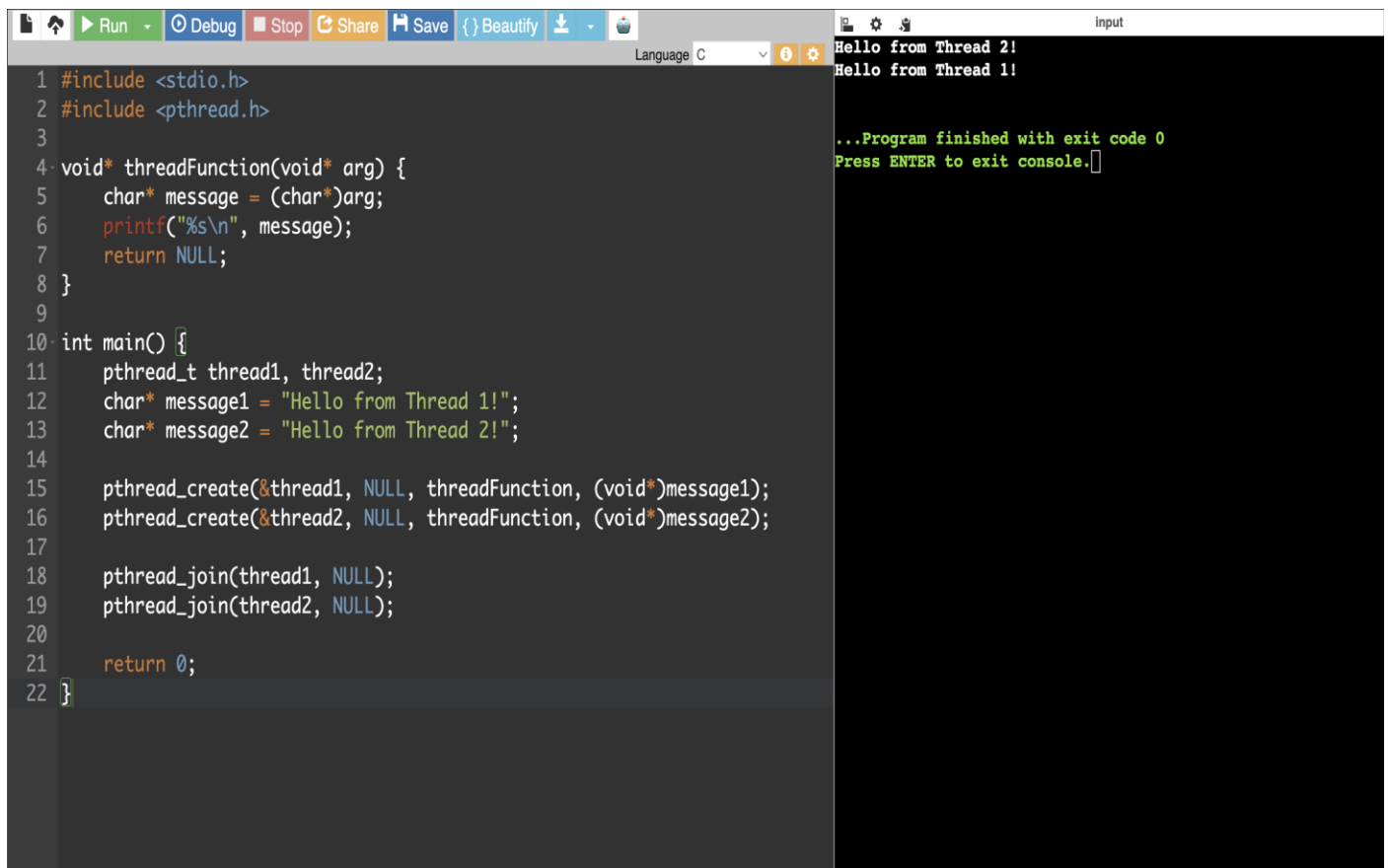
Producer: Data sent to message queue: Hello, message queue!
Consumer: Data received from message queue: Hello, message queue!

Input

Enter your input here...

EXPERIMENT 11:

Illustrate the concept of multithreading using a C program.



```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void* threadFunction(void* arg) {
5     char* message = (char*)arg;
6     printf("%s\n", message);
7     return NULL;
8 }
9
10 int main() {
11     pthread_t thread1, thread2;
12     char* message1 = "Hello from Thread 1!";
13     char* message2 = "Hello from Thread 2!";
14
15     pthread_create(&thread1, NULL, threadFunction, (void*)message1);
16     pthread_create(&thread2, NULL, threadFunction, (void*)message2);
17
18     pthread_join(thread1, NULL);
19     pthread_join(thread2, NULL);
20
21     return 0;
22 }
```

Language: C

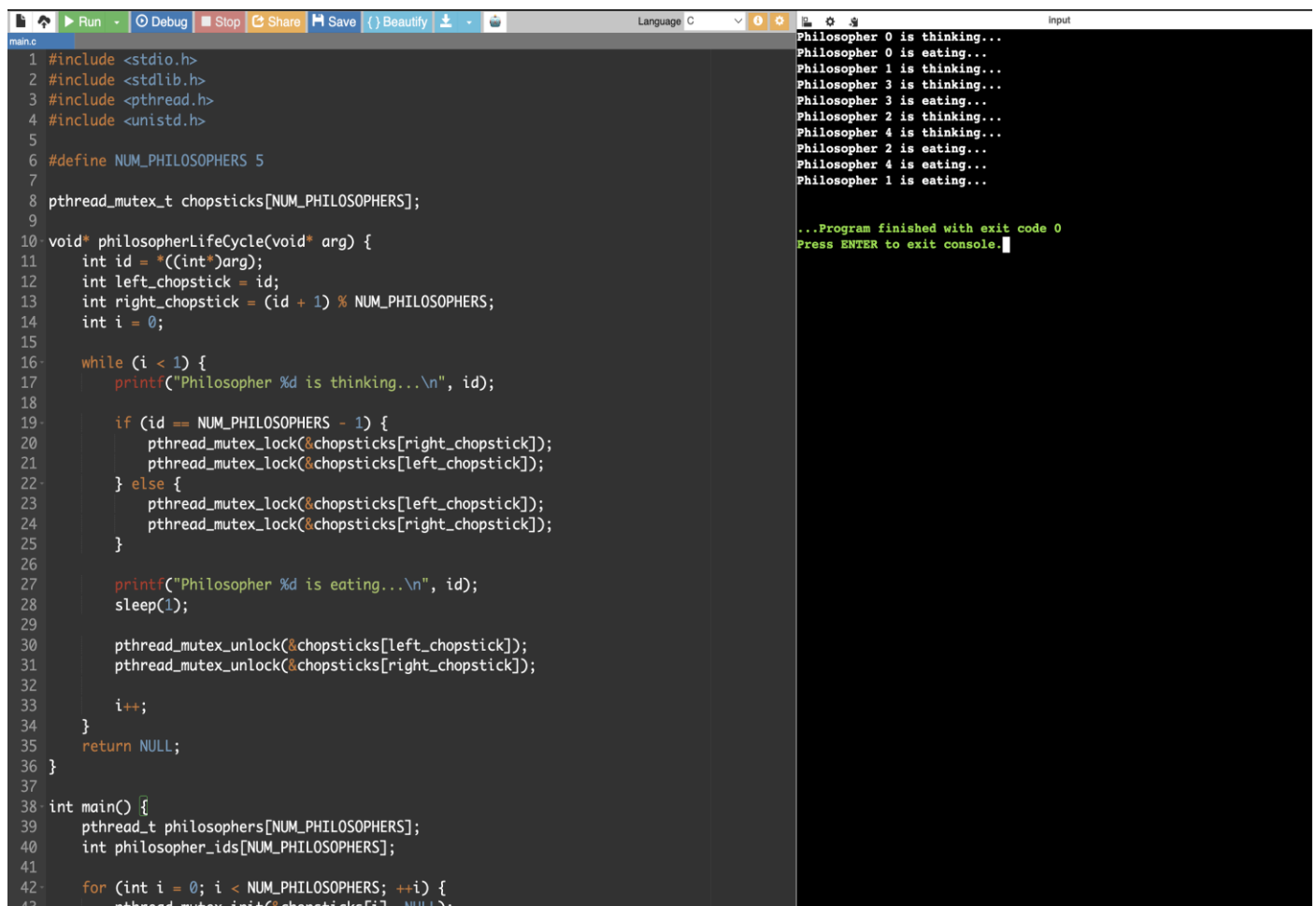
input

Hello from Thread 2!
Hello from Thread 1!

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 12:

Design a C program to simulate the concept of Dining-Philosophers problem



The image shows a code editor window with a C program for simulating the Dining-Philosophers problem. The program is named 'main.c' and is written in C. It defines 5 philosophers and uses pthreads to manage their lifecycle. The output window on the right shows the execution results, where each philosopher alternates between thinking and eating, with a sleep of 1 second between eating and thinking. The program ends with an exit code of 0.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <unistd.h>
5
6 #define NUM_PHILOSOPHERS 5
7
8 pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
9
10 void* philosopherLifeCycle(void* arg) {
11     int id = *((int*)arg);
12     int left_chopstick = id;
13     int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;
14     int i = 0;
15
16     while (i < 1) {
17         printf("Philosopher %d is thinking...\n", id);
18
19         if (id == NUM_PHILOSOPHERS - 1) {
20             pthread_mutex_lock(&chopsticks[right_chopstick]);
21             pthread_mutex_lock(&chopsticks[left_chopstick]);
22         } else {
23             pthread_mutex_lock(&chopsticks[left_chopstick]);
24             pthread_mutex_lock(&chopsticks[right_chopstick]);
25         }
26
27         printf("Philosopher %d is eating...\n", id);
28         sleep(1);
29
30         pthread_mutex_unlock(&chopsticks[left_chopstick]);
31         pthread_mutex_unlock(&chopsticks[right_chopstick]);
32
33         i++;
34     }
35     return NULL;
36 }
37
38 int main() {
39     pthread_t philosophers[NUM_PHILOSOPHERS];
40     int philosopher_ids[NUM_PHILOSOPHERS];
41
42     for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
43         pthread_mutex_init(&chopsticks[i], NULL);
44         philosopher_ids[i] = i;
45         pthread_create(&philosophers[i], NULL, philosopherLifeCycle, &philosopher_ids[i]);
46     }
47
48     for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
49         pthread_join(philosophers[i], NULL);
50     }
51
52     return 0;
53 }
```

Philosopher 0 is thinking...
Philosopher 0 is eating...
Philosopher 1 is thinking...
Philosopher 3 is thinking...
Philosopher 3 is eating...
Philosopher 2 is thinking...
Philosopher 4 is thinking...
Philosopher 2 is eating...
Philosopher 4 is eating...
Philosopher 1 is eating...

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 13:

Construct a C program for implementation of the various memory allocation Strategies

```
1 #include <stdio.h>
2
3 void bestfit(int mp[], int p[], int m, int n) {
4     int j = 0;
5     for (int i = 0; i < n; i++) {
6         if (j >= m) {
7             break;
8         }
9         if (mp[i] >= p[j]) {
10             printf("\n%d fits in %d", p[j], mp[i]);
11             mp[i] = mp[i] - p[j];
12             j++;
13             i = i - 1;
14         }
15     }
16     for (int i = j; i < m; i++) {
17         printf("\n%d must wait for its process", p[i]);
18     }
19     printf("\n");
20 }
21
22 void rsort(int a[], int n) {
23     for (int i = 0; i < n; i++) {
24         for (int j = 0; j < n; j++) {
25             if (a[i] > a[j]) {
26                 int t = a[i];
27                 a[i] = a[j];
28                 a[j] = t;
29             }
30         }
31     }
32 }
33
34 void sort(int a[], int n) {
35     for (int i = 0; i < n; i++) {
36         for (int j = 0; j < n; j++) {
37             if (a[i] < a[j]) {
38                 int t = a[i];
39                 a[i] = a[j];
40                 a[j] = t;
41             }
42         }
43     }
44 }
45
46 void firstfit(int mp[], int p[], int m, int n) {
47     sort(mp, n);
48     sort(p, m);
49     bestfit(mp, p, m, n);
50 }
```

Status: **Accepted**

Memory Used: 728 byte Execution Time: 0.003 sec

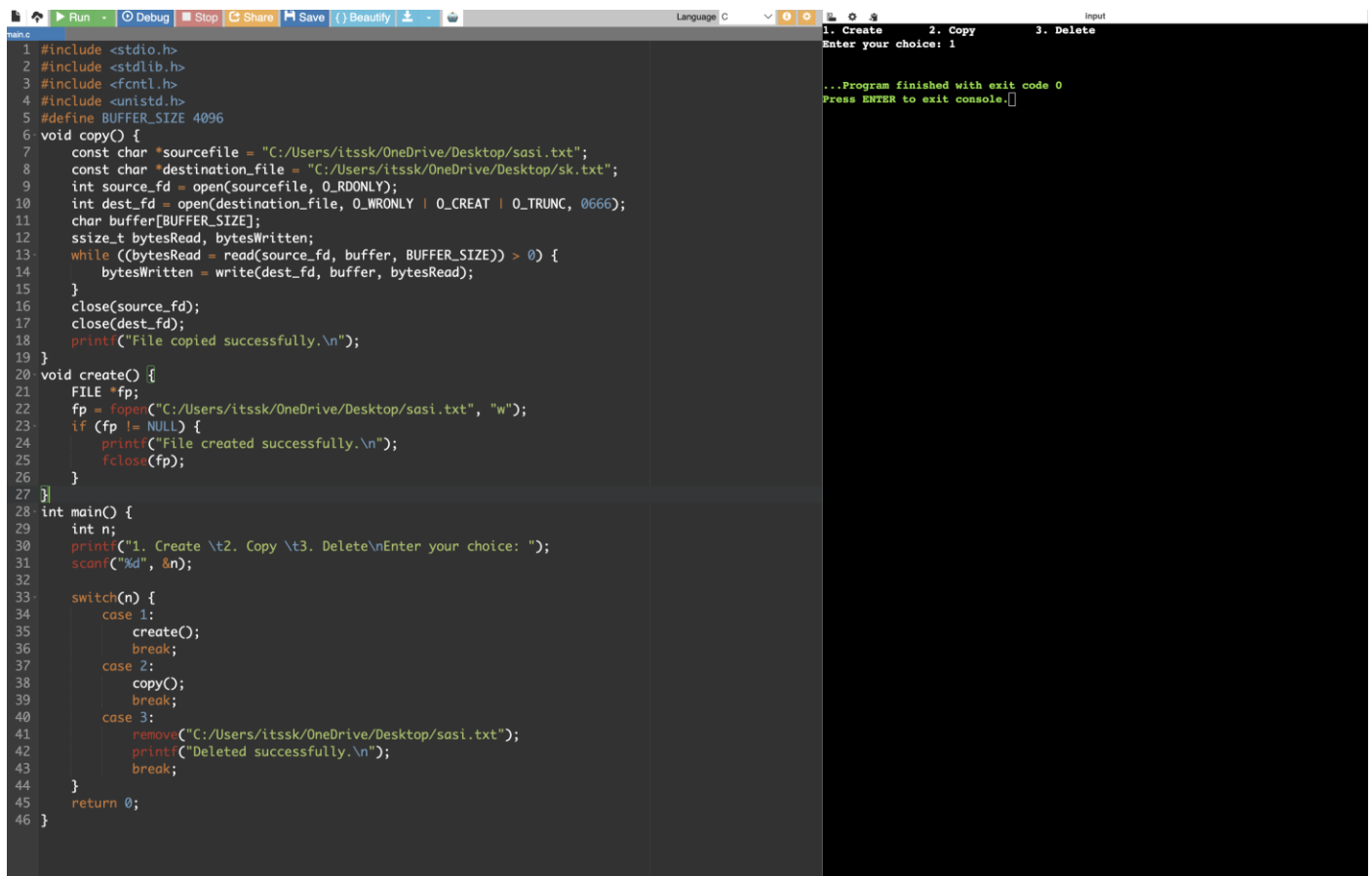
Number of memory partition : Number of process : Enter the memory partitions :
Enter process size :
1. Firstfit 2. Bestfit 3. worstfit
Enter your choice :
212 fits in 500
417 must wait for its process
112 must wait for its process

Input

3
3
100
500
200
212
417
112
1

EXPERIMENT 14:

Construct a C program to organise the file using a single level directory.



The image shows a C program in a code editor and its execution in a console. The code defines three functions: `copy()`, `create()`, and `main()`. `copy()` copies the contents of `sasi.txt` to `sk.txt`. `create()` creates `sasi.txt` if it doesn't exist. `main()` prompts the user to choose between creating, copying, or deleting the file. The console shows the user entering '1' to create the file, and the program finishing with exit code 0.

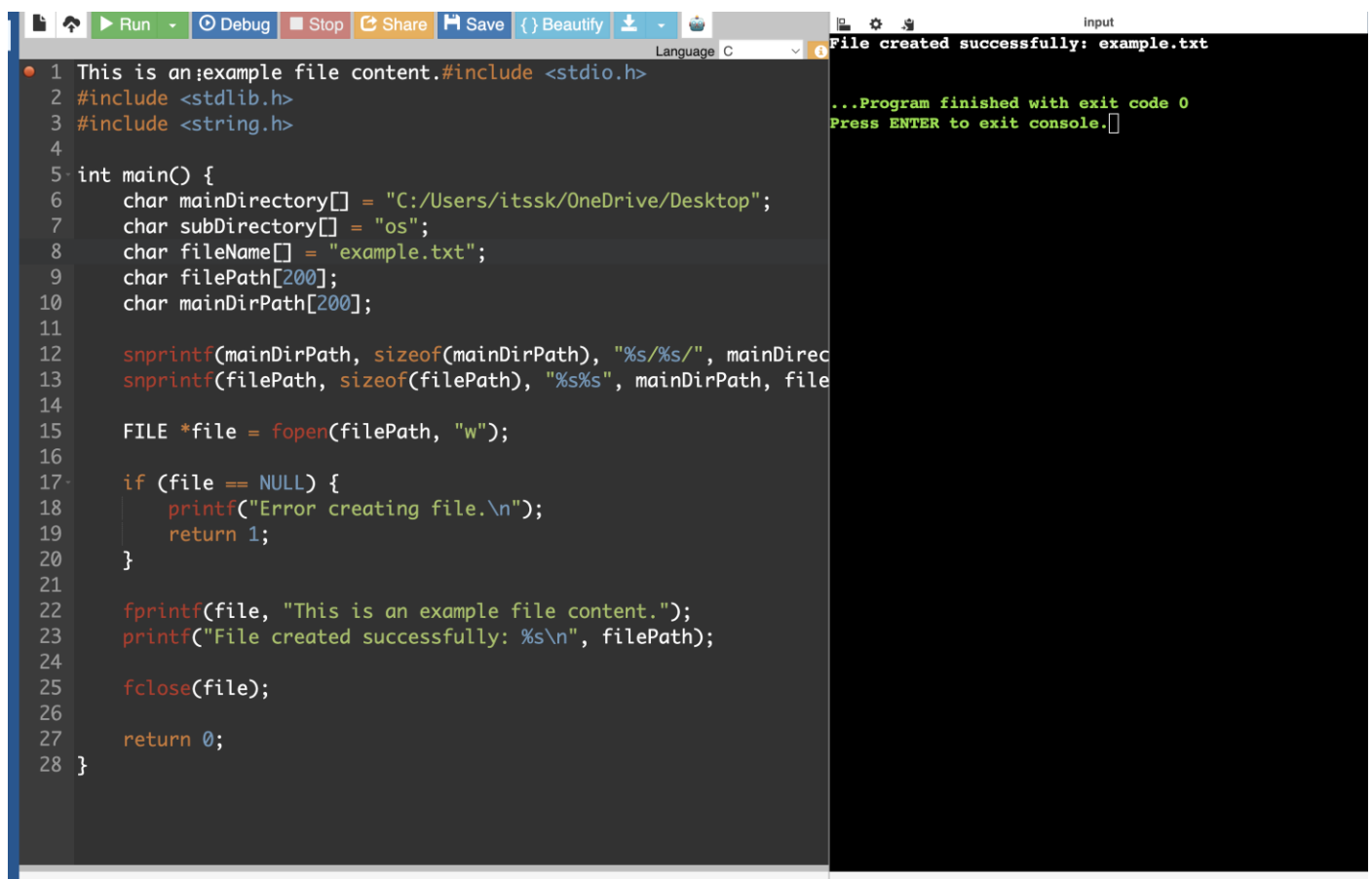
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #define BUFFER_SIZE 4096
6 void copy() {
7     const char *sourcefile = "C:/Users/itssk/OneDrive/Desktop/sasi.txt";
8     const char *destination_file = "C:/Users/itssk/OneDrive/Desktop/sk.txt";
9     int source_fd = open(sourcefile, O_RDONLY);
10    int dest_fd = open(destination_file, O_WRONLY | O_CREAT | O_TRUNC, 0666);
11    char buffer[BUFFER_SIZE];
12    ssize_t bytesRead, bytesWritten;
13    while ((bytesRead = read(source_fd, buffer, BUFFER_SIZE)) > 0) {
14        bytesWritten = write(dest_fd, buffer, bytesRead);
15    }
16    close(source_fd);
17    close(dest_fd);
18    printf("File copied successfully.\n");
19 }
20 void create() {
21     FILE *fp;
22     fp = fopen("C:/Users/itssk/OneDrive/Desktop/sasi.txt", "w");
23     if (fp != NULL) {
24         printf("File created successfully.\n");
25         fclose(fp);
26     }
27 }
28 int main() {
29     int n;
30     printf("1. Create \t2. Copy \t3. Delete\nEnter your choice: ");
31     scanf("%d", &n);
32
33     switch(n) {
34         case 1:
35             create();
36             break;
37         case 2:
38             copy();
39             break;
40         case 3:
41             remove("C:/Users/itssk/OneDrive/Desktop/sasi.txt");
42             printf("Deleted successfully.\n");
43             break;
44     }
45     return 0;
46 }
```

```
1. Create      2. Copy      3. Delete
Enter your choice: 1

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 15:

Design a C program to organise the file using a two level directory structure.



The image shows a code editor window with a C program and a console window displaying the program's output. The code editor has a menu bar with options: Run, Debug, Stop, Share, Save, Beautify, and a download icon. The language is set to C. The code is as follows:

```
1 This is an example file content. #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     char mainDirectory[] = "C:/Users/itssk/OneDrive/Desktop";
7     char subDirectory[] = "os";
8     char fileName[] = "example.txt";
9     char filePath[200];
10    char mainDirPath[200];
11
12    snprintf(mainDirPath, sizeof(mainDirPath), "%s/%s/", mainDirectory, subDirectory);
13    snprintf(filePath, sizeof(filePath), "%s%s", mainDirPath, fileName);
14
15    FILE *file = fopen(filePath, "w");
16
17    if (file == NULL) {
18        printf("Error creating file.\n");
19        return 1;
20    }
21
22    fprintf(file, "This is an example file content.");
23    printf("File created successfully: %s\n", filePath);
24
25    fclose(file);
26
27    return 0;
28 }
```

The console window shows the following output:

```
File created successfully: example.txt
...Program finished with exit code 0
Press ENTER to exit console.
```